# Lecture 8: Virtual Memory & Memory Management

Introduction to Computer Systems

Instructors: Wei Xu, Mingyu Gao
Teaching Assistants: Weitao Wan, Tingrui Zhang, Ruitao Liu, Shuo Chen

Spring 2026

## Contents

**1 Introduction** **3**

**2 Process Address Spaces and the Limits of Base-and-Bound** **3**
2.1 The address space of a process . . . 3
2.2 Simple base-and-bound protection . . . 3

**3 Virtual and Physical Address Spaces** **5**
3.1 Two names for memory locations . . . 5
3.2 Page offset and page number . . . 5

**4 Page Tables and Page Table Entries** **6**
4.1 The page table as the translation data structure . . . 6
4.2 Single-level translation algorithm . . . 6

**5 Why Virtual Memory?** **7**
5.1 Virtual memory as memory management . . . 7
5.2 Virtual memory as protection . . . 7
5.3 Virtual memory as data caching . . . 9

**6 Demand Paging and Page Faults** **9**
6.1 Page states . . . 9
6.2 Handling a page fault . . . 10
6.3 Locality, working sets, and thrashing . . . 10

**7 Speeding Up Translation with the TLB** **11**
7.1 Translation locality . . . 11
7.2 TLB organization . . . 12
7.3 TLB misses and page table walks . . . 12
7.4 Complete translation flow . . . 13
7.5 Virtual memory and the cache hierarchy . . . 13

**8 The Page Table Size Problem** **14**
8.1 Why a flat page table is too large . . . 14
8.2 Sparse representation with multi-level page tables . . . 14
8.3 x86-64 four-level translation . . . 15
8.4 Space savings and performance cost . . . 15

**9 Sharing, Copy-on-Write, and Memory Mapping** **16**
9.1 Inter-process page sharing . . . 16
9.2 Copy-on-write . . . 17
9.3 Memory mapping . . . 17
9.4 Revisiting `fork` and `execve` . . . 17
9.5 User-level `mmap` and `munmap` . . . 18

**10 Dynamic Memory Allocation** **19**
10.1 The heap and the allocator . . . 19
10.2 Allocator API . . . 20
10.3 Allocator internals . . . 20
10.4 External fragmentation . . . 21
10.5 Segregated free lists . . . 21
10.6 Internal fragmentation . . . 22

**11 Implicit Memory Management and Garbage Collection** **22**
11.1 Why explicit memory management is error-prone . . . 22
11.2 Reachability and pointer discipline . . . 24
11.3 Memory as a graph . . . 24
11.4 Mark-and-sweep garbage collection . . . 24
11.5 Reference counting . . . 26

**12 Summary** **26**

**13 Quizzes & Exercises** **27**
13.1 Short-Answer Quiz . . . 27
13.2 Essay Questions . . . 27
13.3 Additional Practice Problems . . . 27
13.4 Answer Key for the Short-Answer Quiz . . . 28

# 1 Introduction

Modern computer systems must make one physical machine serve many programs safely and efficiently. A laptop, server, or smartphone has a finite amount of DRAM, yet it may run hundreds or thousands of processes. Each process would like to behave as if it owns a simple linear memory array: code at some addresses, global data at others, a heap that grows as dynamic data structures are created, and a stack that grows as functions call each other. The operating system must turn this convenient illusion into reality while also preventing one program from reading or corrupting another program's data.

Virtual memory is the central mechanism that accomplishes this. It introduces a layer of *indirection*: programs issue *virtual addresses*, while the hardware ultimately accesses *physical addresses*. A mapping maintained by the operating system and consulted by the processor translates between the two. This one idea supports flexible memory allocation, process isolation, page sharing, demand paging from disk or SSD, copy-on-write process creation, memory-mapped files, and modern dynamic allocation.

This lecture studies virtual memory from the hardware-software boundary upward. We begin with why a simple base-and-bound address space is insufficient, then build page-based translation, page tables, page faults, demand paging, the translation look-aside buffer (TLB), multi-level page tables, inter-process sharing, memory mapping, dynamic heap allocation, fragmentation, and garbage collection.

Definition

**Virtual memory** is a mechanism that gives each process a private, linear virtual address space and maps its virtual pages to physical memory frames or to backing storage. The mapping is controlled by the operating system and used by hardware during every memory access.

# 2 Process Address Spaces and the Limits of Base-and-Bound

## 2.1 The address space of a process

A process is the operating system's representation of a running program. It contains one or more threads of execution and an address space. The address space is the set of memory locations that the process can name. At the programming-language level, this memory is usually described in regions:

- **Text/code**: machine instructions, usually read-only and executable.
- **Static data**: global and static variables, including initialized and zero-initialized data.
- **Heap**: dynamically allocated objects managed by an allocator such as `malloc` or C++ `new`.
- **Stack**: call frames, local variables, return addresses, and saved registers.
- **Kernel mapping**: privileged kernel memory, typically protected from user-mode access.

The important point is that all processes share the same physical DRAM chips, but they must not share memory arbitrarily. Sharing must be intentional and controlled; isolation must be the default.

## 2.2 Simple base-and-bound protection

A very simple way to implement address spaces is to divide physical memory into contiguous regions and assign one region to each process. The processor maintains two privileged registers:

- **Base**: the starting physical address of the process's memory region.

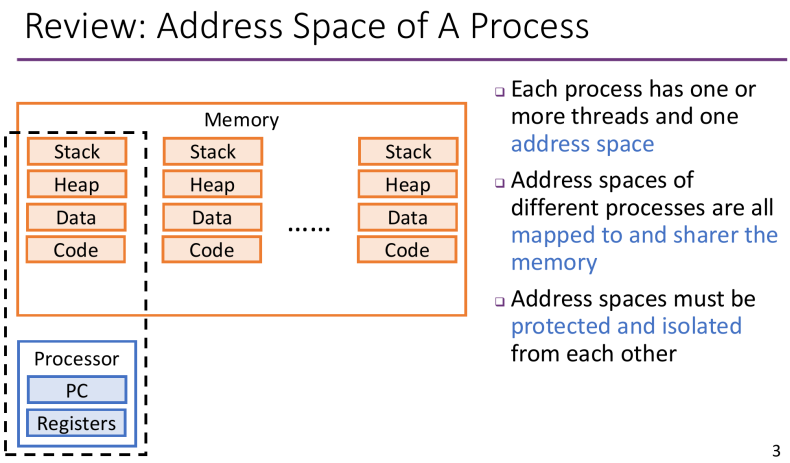



**Figure 1:** A process has an address space and one or more threads. Multiple processes' address spaces are ultimately mapped onto shared physical memory, but protection must ensure that one process cannot directly access another process's private code, heap, stack, or data.

- **Bound**: the size or end of the region.

For every user address $a$, hardware checks whether $0 \le a <$ bound. If the check succeeds, it computes PA $=$ base $+ a$; otherwise, it raises an exception. This provides a basic form of relocation and protection.

$$\text{PA} = \begin{cases} \text{base} + \text{VA}, & 0 \le \text{VA} < \text{bound}, \\ \text{exception}, & \text{otherwise}. \end{cases} \tag{1}$$

This mechanism is easy to understand, but it is too rigid for modern systems.

- **Contiguity**: each process needs a physically contiguous region even if physical memory has enough free space scattered in many pieces.
- **Dynamic growth**: if the heap or stack grows, the process may collide with the neighboring process's region.
- **Sharing**: two processes cannot easily share a selected region, such as a shared library or communication buffer, without giving up the simplicity of one contiguous private range.

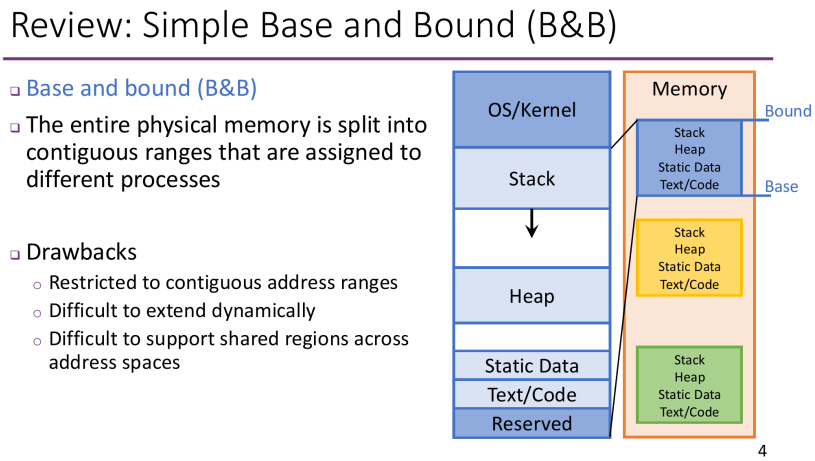



**Figure 2:** Base-and-bound divides physical memory into large contiguous ranges. It is simple but makes dynamic growth, sparse allocation, and selective sharing difficult.

# 3 Virtual and Physical Address Spaces

## 3.1 Two names for memory locations

Virtual memory separates the name used by a program from the location used by the hardware. A process emits virtual addresses. DRAM is indexed by physical addresses. Translation maps the former to the latter.

**Definition**

A **virtual address space** is the private, linear address space visible to a process. A **physical address space** is the single hardware address space corresponding to installed main memory. **Address translation** maps a virtual address to a physical address.

For an $n$-bit virtual address, the virtual address space can contain $2^n$ byte addresses. In many x86-64 systems, user-level virtual addresses use 48 meaningful bits, giving a huge sparse space of $2^{48}$ bytes. Physical memory is much smaller: a machine may have 32 GiB, 64 GiB, or a few hundred GiB of installed DRAM.

The mapping granularity is a **page**. A common page size is $4\,\text{KiB} = 2^{12}$ bytes, although many systems also support huge pages such as 2 MiB or 1 GiB. A physical page is often called a **frame**.

## 3.2 Page offset and page number

For a 4 KiB page, the lowest 12 bits of an address are the *page offset*. These bits identify a byte within the page and are not translated. The remaining high bits form the *virtual page number* (VPN). Translation replaces the VPN with a *physical frame number* (PFN), while preserving the offset.

$$\text{VA} = (\text{VPN}, \text{offset}), \qquad \text{PA} = (\text{PFN}, \text{offset}). \tag{2}$$

This design preserves spatial structure within a page: byte 100 of a virtual page becomes byte 100 of the corresponding physical frame. However, adjacent virtual pages need not be adjacent in physical memory.

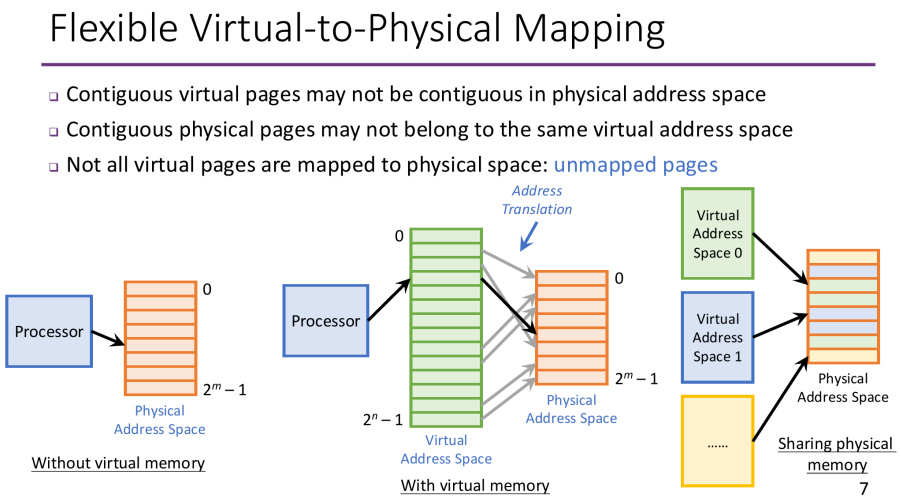



**Figure 3:** Virtual pages can be mapped flexibly to physical frames. Contiguous virtual pages need not be contiguous in physical memory, contiguous physical frames may belong to unrelated processes, and some virtual pages may be unmapped.

# 4 Page Tables and Page Table Entries

## 4.1 The page table as the translation data structure

The page table is an operating-system data structure used by the hardware to translate virtual pages into physical frames. Conceptually, a simple page table is an array indexed by VPN. Each entry describes one virtual page.

A page table entry (PTE) normally contains:

- a **valid** or **present** bit indicating whether the virtual page is currently mapped to a physical frame;
- the **physical frame number** if the page is present;
- permission bits, often **R**, **W**, and **X**, controlling read, write, and execute access;
- status bits such as **accessed/reference** and **dirty**, useful for replacement and write-back decisions;
- information about backing storage, such as a swap slot or file offset, when the page is allocated but not resident in DRAM.

The operating system creates and modifies page tables. The processor consults them during memory accesses. This split is crucial: the operating system controls the mapping policy, while hardware performs the common case quickly.

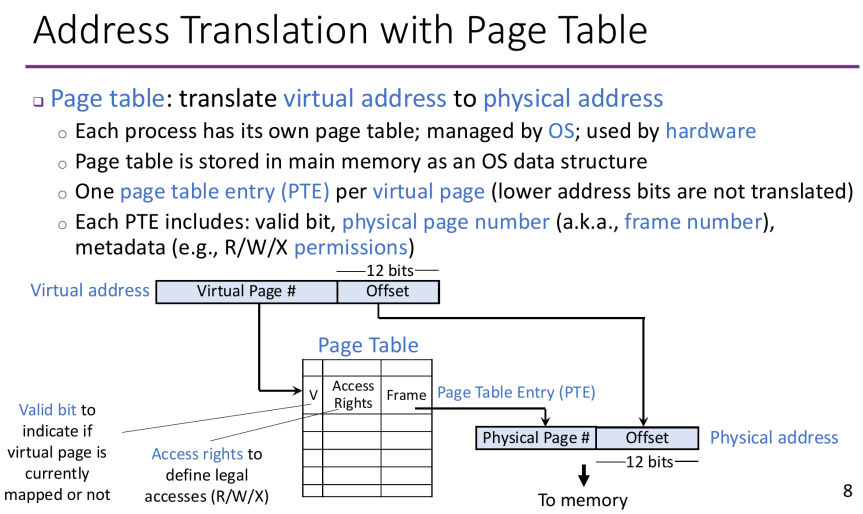



**Figure 4:** A virtual address is divided into a virtual page number and page offset. The virtual page number indexes the page table. The PTE supplies validity, access rights, and the physical page number; the offset is appended unchanged to form the physical address.

## 4.2 Single-level translation algorithm

The following algorithm abstracts the translation process before adding TLBs and multi-level page tables.

**Algorithm 1:** Single-Level Page Table Translation

**Input:** Virtual address VA, access type $a \in \{\text{read}, \text{write}, \text{execute}\}$, page table $PT$
**Output:** Physical address PA or an exception

VPN $\leftarrow \lfloor \text{VA}/\text{page_size} \rfloor$;
$offset \leftarrow \text{VA} \bmod \text{page_size}$;
$pte \leftarrow PT[\text{VPN}]$;
**if** $pte.valid = 0$ **then**
    **return** page fault;
**if** *$a$ is not permitted by $pte.RWX$* **then**
    **return** permission fault;
PA $\leftarrow pte.PFN \times \text{page_size} + offset$;
**return** PA;

Several important consequences follow immediately. First, the processor cannot simply access arbitrary physical memory; it can only reach frames named by valid PTEs in the current process's page table. Second, the same virtual address can map to different physical frames in different processes, because each process has its own page table. Third, two processes can deliberately share memory by placing the same PFN in their page tables.

# 5 Why Virtual Memory?

Virtual memory is not merely a way to make addresses larger. It simultaneously solves three system problems: memory management, protection, and caching.

## 5.1 Virtual memory as memory management

With virtual memory, each process sees a simple contiguous virtual address space, while the operating system can place its pages anywhere in physical memory. Allocating a new virtual page requires finding a free physical frame, filling or zeroing it as needed, and installing a PTE. If the heap grows, the operating system can map additional pages without physically moving the whole process. If a process releases memory, the frames can be returned to a global free list.

This eliminates the base-and-bound contiguity problem. A process's virtual heap may be contiguous even if its physical frames are scattered throughout DRAM.

## 5.2 Virtual memory as protection

Protection comes from two facts. First, a user process cannot name physical addresses directly; its addresses must pass through the page table. Second, the PTE includes permissions that hardware checks on every access.

For example, executable code pages are often marked read-execute but not writable. Stack and heap pages are normally read-write but not executable. Kernel pages may be absent from a user page table or present but marked supervisor-only, so user-mode accesses trap. These policies help enforce both process isolation and system integrity.

**Deep Dive**

The familiar "segmentation fault" in C/C++ is usually the visible result of a hardware exception raised during address translation. The underlying cause may be an unmapped page, a write to a read-only page, an execute attempt on non-executable memory, or an access to a kernel-only page. The exception transfers control to the kernel, which either repairs the situation, as in demand paging or copy-on-write, or terminates the process.

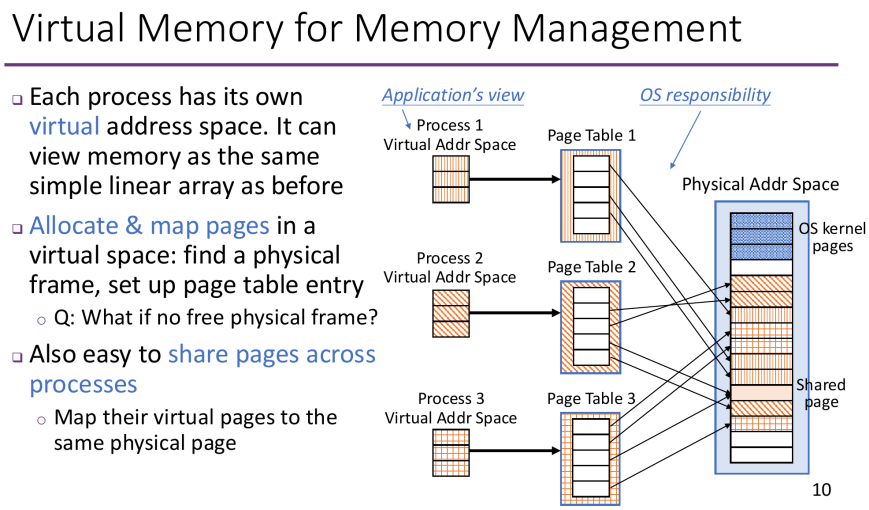



**Figure 5:** Virtual memory separates the application's view from the operating system's responsibility. Each process sees a private virtual address space; the OS maps pages to physical frames and can also map virtual pages from different processes to the same physical frame for sharing.

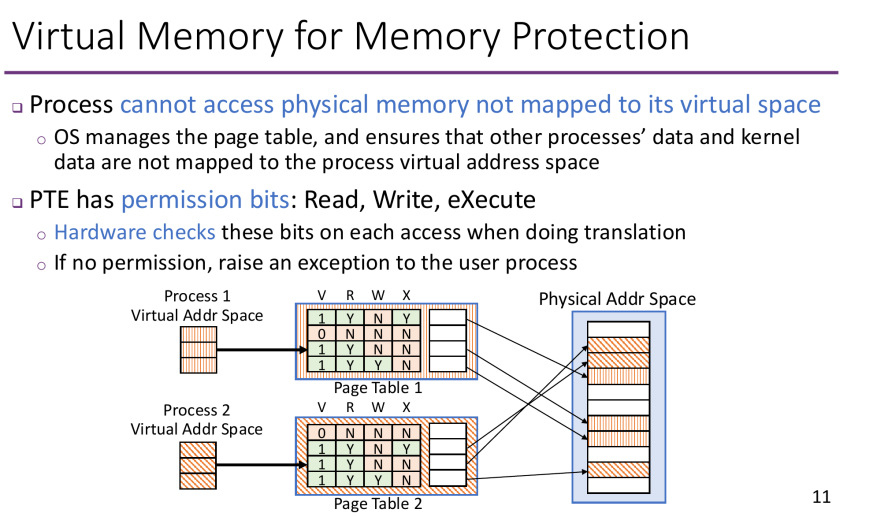



**Figure 6:** Page table permissions implement fine-grained protection. Even when a page is mapped, read, write, and execute permission bits determine which operations are legal. Unauthorized operations raise exceptions.

### 5.3 Virtual memory as data caching

A process's allocated virtual address space may exceed physical DRAM capacity, especially when many processes run at once. Therefore, not every allocated page can always be resident in memory. Some pages reside in DRAM; others are stored on disk or SSD in swap space or in ordinary files. In this sense, DRAM is a cache for a larger backing store.

The analogy with processor caches is useful but not identical:

- processor caches use small blocks such as 64-byte cache lines; virtual memory uses much larger blocks such as 4-KiB pages;
- processor caches are usually managed by hardware; paging is managed by the operating system;
- a cache miss costs tens or hundreds of cycles; a page fault can cost microseconds to milliseconds because it may involve storage I/O and scheduling.

Since the operating system can place any virtual page in any available physical frame, DRAM-as-a-page-cache is best viewed as fully associative at the page level. The page table, rather than a hardware tag comparison in a fixed cache set, records the current location of each page.

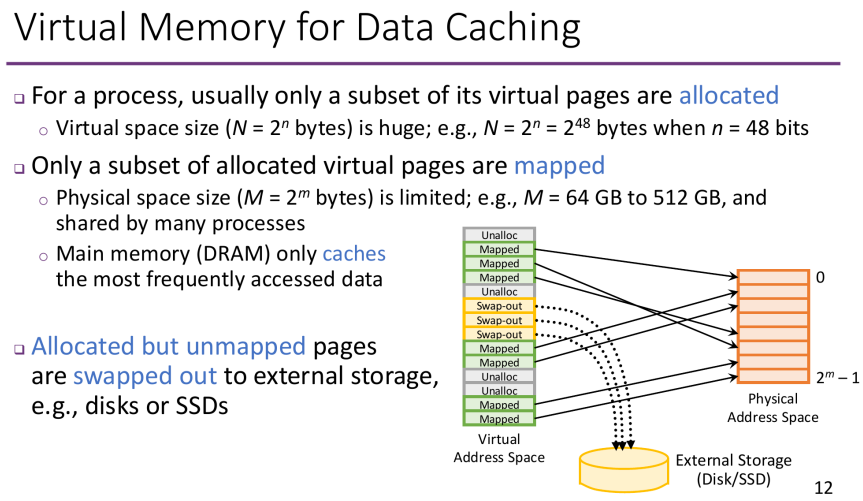



**Figure 7:** Only a subset of allocated virtual pages are resident in physical memory. Allocated pages that are not currently resident are swapped out to external storage such as disk or SSD and are brought back on demand.

## 6 Demand Paging and Page Faults

### 6.1 Page states

A virtual page can be in one of several states:

| State | Meaning |
|---|---|
| Unallocated | The program has no valid object or mapping there. Access is a program error. |
| Allocated and resident | The PTE is valid/present and contains a PFN. The page can be accessed after permission checks. |
| Allocated but nonresident | The page has a backing location, such as swap or a mapped file, but no current physical frame. Access raises a page fault. |
| Mapped but not permitted | The page exists, but the requested operation violates R/W/X permissions. Access raises a permission fault. |

A page fault is not always a bug. It is a mechanism. When a valid allocated page is not resident, the fault allows the operating system to bring the page into memory on demand.

## 6.2 Handling a page fault

A page fault begins as a hardware exception. The processor saves enough state to resume the faulting instruction later and transfers control to the kernel's page fault handler. The kernel then determines whether the fault is legal and repairable.

**Algorithm 2:** Demand Paging Page Fault Handler

**Input:** Faulting virtual address VA, access type $a$, current process $P$
**Output:** Resume execution or terminate $P$

```
Locate the virtual memory area containing VA;
if no such area exists or a violates the area's permissions then
    terminate P or deliver a signal;
    return error;
if a free physical frame is unavailable then
    choose a victim frame using the page replacement policy;
    if victim is dirty then
        write victim page to its backing store;
    invalidate the victim's PTE and any stale TLB entry;
allocate a physical frame f;
if VA is backed by a file or swap slot then
    read the needed page into f;
else
    fill f with zeros or prepare a copy-on-write page;
update P's PTE for VA to point to f with correct permissions;
retry the faulting instruction;
```

The retry is essential. The original load, store, or instruction fetch failed because the translation was not ready. After the handler updates the page table, executing the same instruction again can succeed.

## 6.3 Locality, working sets, and thrashing

Virtual memory would be unusably slow if every access caused a page table memory access and many accesses caused storage I/O. It works because programs exhibit locality.

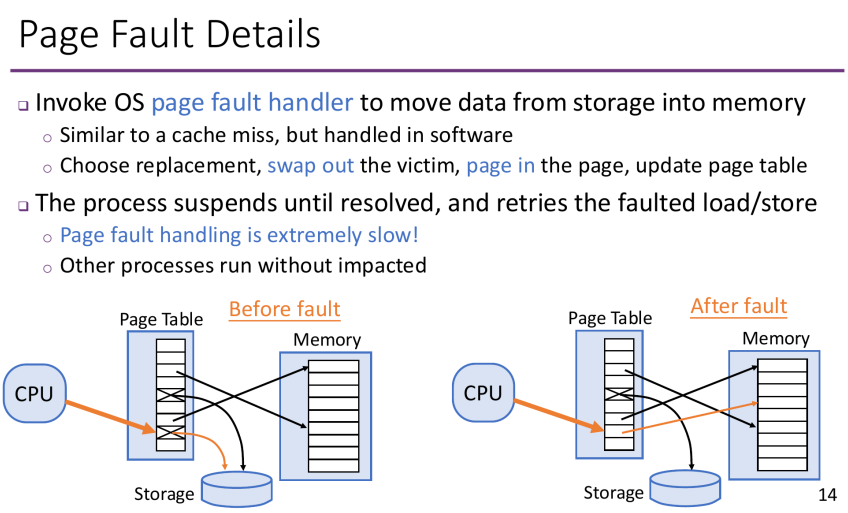



**Figure 8:** On a page fault, the CPU traps to the operating system. The OS locates the data on storage, possibly evicts another page, brings the requested page into memory, updates the page table, and allows the process to retry the access.

**Definition**

A program's **working set** is the set of pages it actively uses over a time interval. If the working set fits in physical memory, page faults are infrequent. If it does not fit, the system may **thrash**: it spends most of its time paging rather than executing useful instructions.

Temporal locality means that recently accessed pages are likely to be accessed again. Spatial locality means that nearby addresses are likely to be accessed soon, which is especially relevant because one page fault brings in an entire page. When locality fails, performance can collapse dramatically. Increasing DRAM capacity or restructuring the program to process data in smaller chunks can reduce working-set pressure.

# 7 Speeding Up Translation with the TLB

## 7.1 Translation locality

A straightforward page table lookup would require a memory access to fetch the PTE before the actual memory access. That would roughly double the number of memory accesses in the common case, and multi-level page tables would be even worse. Processors therefore include a specialized hardware cache for translations: the translation look-aside buffer.

**Definition**

The **translation look-aside buffer (TLB)** is an on-chip hardware cache that stores recently used virtual-page-to-physical-frame translations, along with permission bits and status metadata.

The TLB works because locality in data accesses implies locality in address translations. Repeatedly accessing the same array page requires the same VPN-to-PFN translation. Sequentially scanning memory reuses each translation for many consecutive bytes within the page.

## 7.2 TLB organization

A TLB entry is similar to a cache entry, but its cached data is a PTE or a subset of a PTE. A typical entry contains a valid bit, a VPN tag, possibly an address-space identifier (ASID) or process-context tag, the PFN, and permission bits. On a TLB hit, translation is fast. On a miss, the processor must find the PTE in the page table.

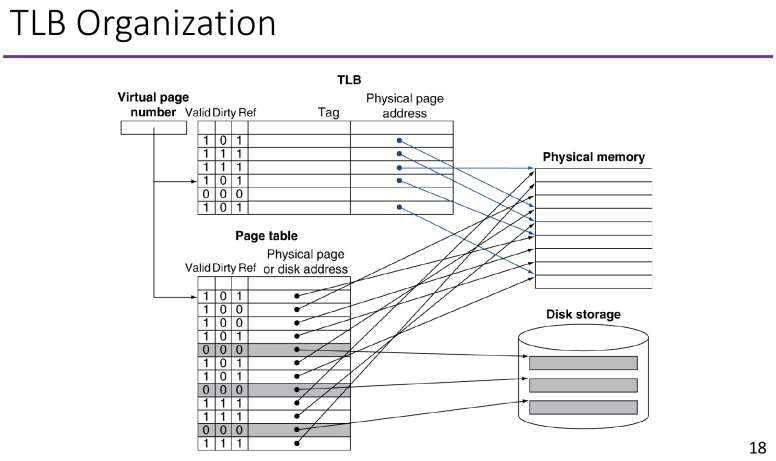



**Figure 9:** The TLB caches selected page table entries. A virtual page number first probes the TLB. If the translation is present, the physical page address is produced immediately; otherwise, the system performs a page table walk. The page table still records whether a page is resident in memory or backed by disk storage.

## 7.3 TLB misses and page table walks

A **TLB miss** is not the same as a **page fault**. A TLB miss means the translation is not currently cached in the TLB. The page itself may still be resident in DRAM. The processor or operating system must perform a **page table walk**: traverse the page table to find the PTE.

There are two common designs:

- **Hardware-managed TLB**: the processor automatically walks the page table on a TLB miss, as in many x86 designs.
- **Software-managed TLB**: the processor raises a special exception, and the operating system fills the TLB, as in some RISC designs.

If the page table walk finds a valid resident page, the translation is copied into the TLB and the instruction is retried. If the PTE indicates that the page is not resident, the system must first resolve a page fault and then fill the TLB.

## 7.4 Complete translation flow

**Algorithm 3:** Address Translation with TLB, Permission Checks, and Demand Paging

**Input:** Virtual address VA, access type $a$
**Output:** Physical address PA or exception handling result

```
Probe the TLB using the virtual page number of VA;
if TLB hit then
    obtain cached PFN and permissions;
else
    perform a page table walk;
    if PTE indicates page is not allocated then
        return page fault error;
    if PTE indicates allocated but nonresident page then
        run demand paging handler;
        retry translation;
    fill the TLB with the resident PTE;
    retry translation;
if a violates cached permissions then
    return permission fault;
combine PFN with page offset to produce PA;
return PA;
```

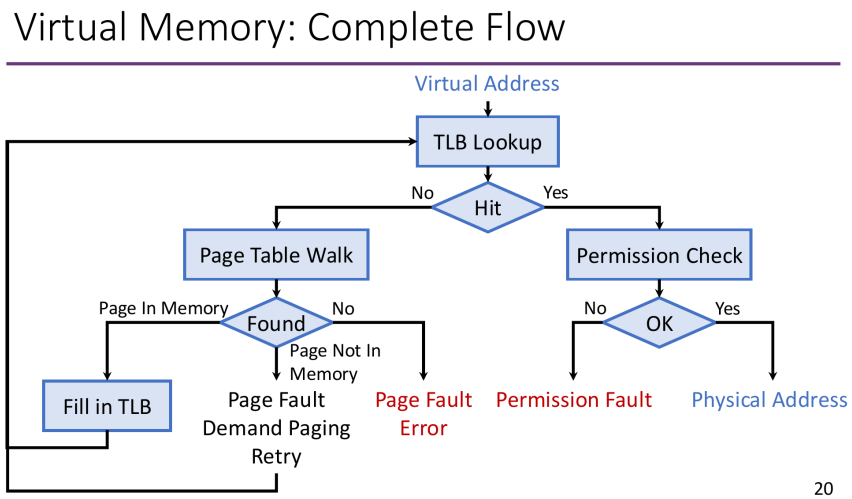



**Figure 10:** The complete virtual-memory flow begins with a TLB lookup. A hit proceeds to permission checking and physical address generation. A miss triggers a page table walk. Depending on the PTE, the system either fills the TLB, handles demand paging, reports a page fault error, or reports a permission fault.

## 7.5 Virtual memory and the cache hierarchy

After translation produces a physical address, the ordinary cache hierarchy proceeds as before: L1, L2, L3, DRAM, and possibly storage if a page fault occurs. Many systems use physically indexed and physically tagged data caches, so cache identity is based on physical addresses rather than virtual aliases.

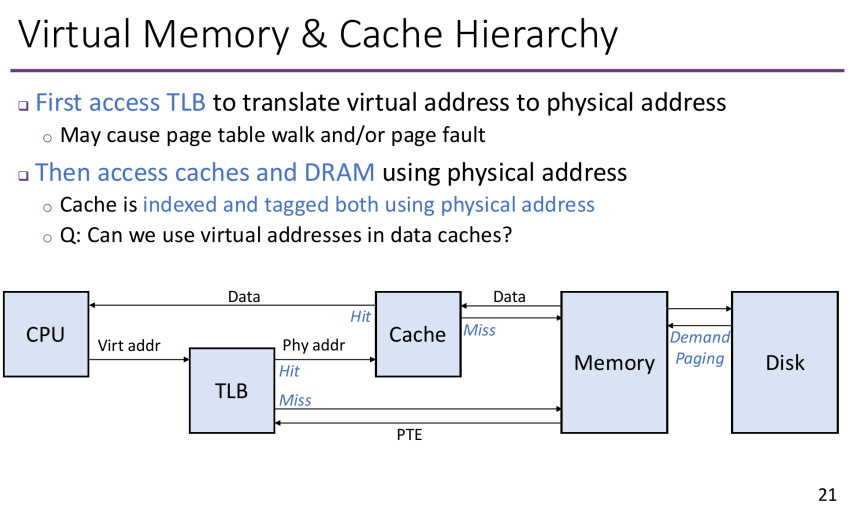



**Figure 11:** A CPU first translates a virtual address through the TLB and, on a TLB miss, through the page table. Once the physical address is known, the cache hierarchy is accessed using that physical address. Demand paging connects DRAM to disk or SSD when the page is not resident.

**Deep Dive**

Why not use virtual addresses directly in data caches? The main difficulty is aliasing. Two processes can map different virtual addresses to the same physical page, for example when sharing memory. A virtually addressed cache might store two cache entries for what should be one memory location. If one process writes through one virtual alias, the other alias may become stale. Adding process identifiers helps distinguish homonyms, where the same virtual address refers to different physical pages, but it does not fully solve synonyms, where different virtual addresses refer to the same physical page. Physically tagged caches avoid this by defining equality in terms of physical identity.

# 8 The Page Table Size Problem

## 8.1 Why a flat page table is too large

A flat page table has one PTE for every virtual page. This is impractical when the virtual address space is huge and sparse. Consider a 48-bit virtual address space, 4-KiB pages, and 8-byte PTEs:

$$\text{number of virtual pages} = \frac{2^{48}}{2^{12}} = 2^{36}, \tag{3}$$

$$\text{flat page table size} = 2^{36} \times 8 \text{ bytes} = 2^{39} \text{ bytes} = 512\,\text{GiB}. \tag{4}$$

A single process would need a page table larger than the DRAM capacity of many machines. Multiple processes would make the situation worse. The key observation is sparsity: most virtual pages are unallocated and do not need explicit PTE storage.

## 8.2 Sparse representation with multi-level page tables

A multi-level page table stores only the portions of the translation structure needed for allocated address ranges. It is a radix tree over address bits. Each internal node is a page table page; each chunk of VPN bits selects an entry at one level; an absent subtree represents a large unallocated virtual range.

**Definition**

A **multi-level page table** is a radix-tree representation of a sparse page table. It uses successive chunks of the virtual page number to index successive levels. Empty virtual address ranges need not allocate lower-level page table pages.

Only the root-level page table must always be directly reachable from a processor register or kernel data structure. Lower levels are allocated on demand as the process uses new virtual ranges.

## 8.3 x86-64 four-level translation

A traditional x86-64 four-level page table uses 48-bit virtual addresses. With 4-KiB pages and 8-byte entries, each page table page contains

$$\frac{4096 \text{ bytes}}{8 \text{ bytes/entry}} = 512 = 2^9 \tag{5}$$

entries. Therefore each level consumes 9 bits of the virtual address. The remaining 12 bits are the page offset:

| Bits | 47–39 | 38–30 | 29–21 | 20–12 | 11–0 |
|---|---|---|---|---|---|
| Meaning | PML4 index | PDP index | PD index | PT index | page offset |
| Width | 9 | 9 | 9 | 9 | 12 |

The CR3 register points to the root page map level 4 (PML4). A page table walk reads a PML4 entry, then a page directory pointer entry, then a page directory entry, then a page table entry, finally yielding the PFN for a 4-KiB data page.

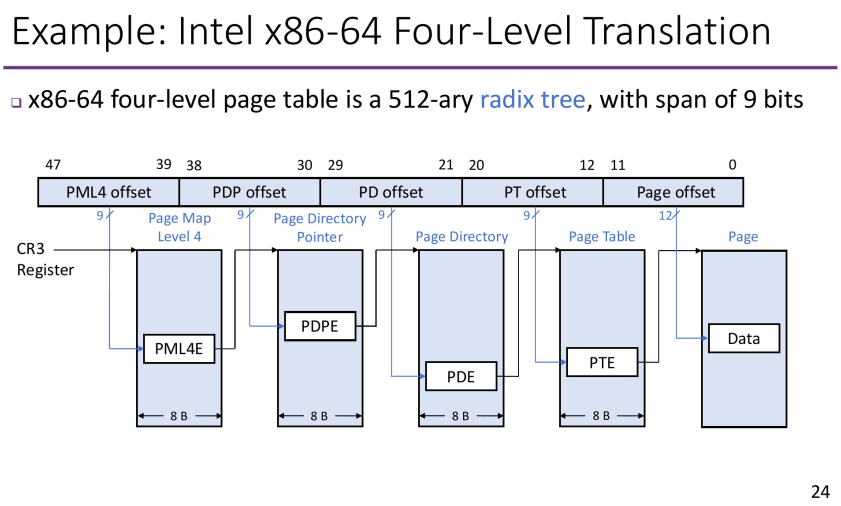



**Figure 12:** x86-64 four-level translation divides the virtual page number into four 9-bit indices. Each index selects one of 512 entries at its level. The CR3 register points to the root, and the final PTE points to the data page.

## 8.4 Space savings and performance cost

If a process uses only low addresses and high kernel-related addresses, most middle virtual ranges are empty. Multi-level page tables allocate nodes only along paths to used pages, avoiding massive empty flat arrays.

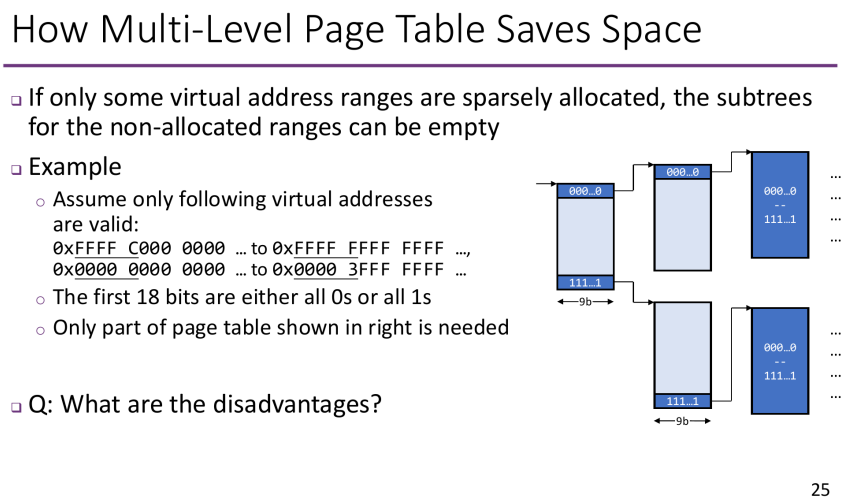



**Figure 13:** Multi-level page tables save space when virtual address usage is sparse. Entire subtrees for unused address ranges can be omitted, so memory overhead is closer to the number of allocated regions than to the full virtual address space size.

The cost is that a TLB miss may require multiple memory references to walk the tree. In the four-level example, a miss can require four page-table memory reads before the actual data access. TLBs, page-walk caches, and huge pages mitigate this cost, but the tradeoff remains: multi-level page tables save memory at the expense of more complex and potentially slower misses.

# 9 Sharing, Copy-on-Write, and Memory Mapping

## 9.1 Inter-process page sharing

Virtual memory makes sharing precise. If two processes should share a page, the operating system installs PTEs in both page tables that point to the same PFN. The virtual addresses can differ. What matters is that the physical frame is the same.

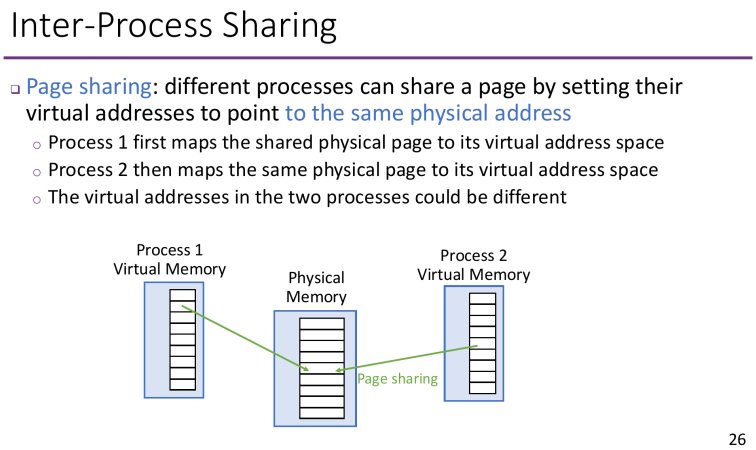



**Figure 14:** Two processes can share a physical page by mapping virtual pages in their distinct address spaces to the same physical frame. The virtual addresses need not match.

Page sharing is used for shared libraries, memory-mapped files, inter-process communication buffers, and kernel mechanisms that avoid unnecessary duplication.

## 9.2 Copy-on-write

Copying a page immediately can waste time and memory if the copy is never modified. Copy-on-write (CoW) delays the physical copy until the first write.

The idea is simple:

1. Make the destination virtual page point to the same physical frame as the source.
2. Mark both mappings read-only, even if the logical object is writable.
3. If either process or virtual page attempts to write, hardware raises a permission fault.
4. The kernel handles the fault by allocating a new frame, copying the data, updating one PTE, restoring write permission, and retrying the write.

Reads see correct copied data without copying. Writes pay the copy cost only when necessary.

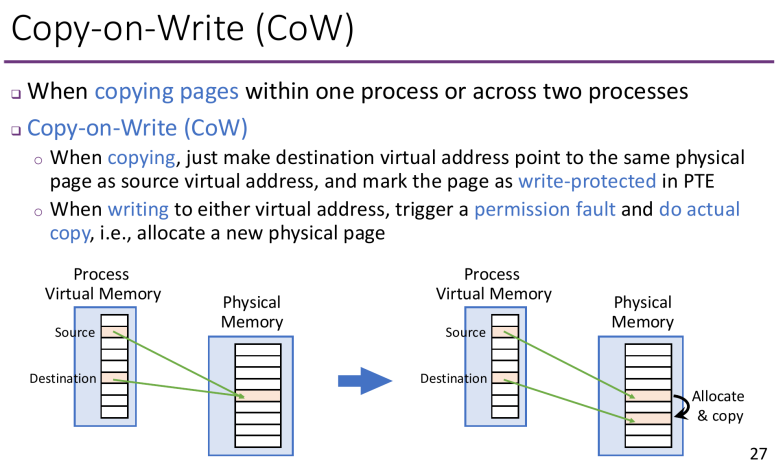



**Figure 15:** Copy-on-write initially maps source and destination virtual pages to the same physical frame and marks them write-protected. The first write triggers a permission fault, causing the OS to allocate a new frame and perform the actual copy.

## 9.3 Memory mapping

Memory mapping associates a virtual memory region with a file-like backing object. The backing object may be:

- a regular file, such as an executable or data file;
- a swap file or swap partition used to store evicted anonymous pages;
- an anonymous zero-filled object used for newly allocated heap or stack pages.

At mapping time, the operating system can avoid copying file data into DRAM. It records the association in its virtual memory metadata and page table state. The first actual access raises a page fault, and demand paging brings in the data. This lazy transfer is another example of delaying expensive work until it is needed.

## 9.4 Revisiting `fork` and `execve`

Virtual memory explains why process creation can be efficient.

**fork.** The child process should initially see the same address-space contents as the parent, but it must become independent after either process writes. Rather than copying every data page, the kernel copies the page table, points both page tables to the same frames, and marks writable pages copy-on-write. Subsequent writes create private copies page by page.

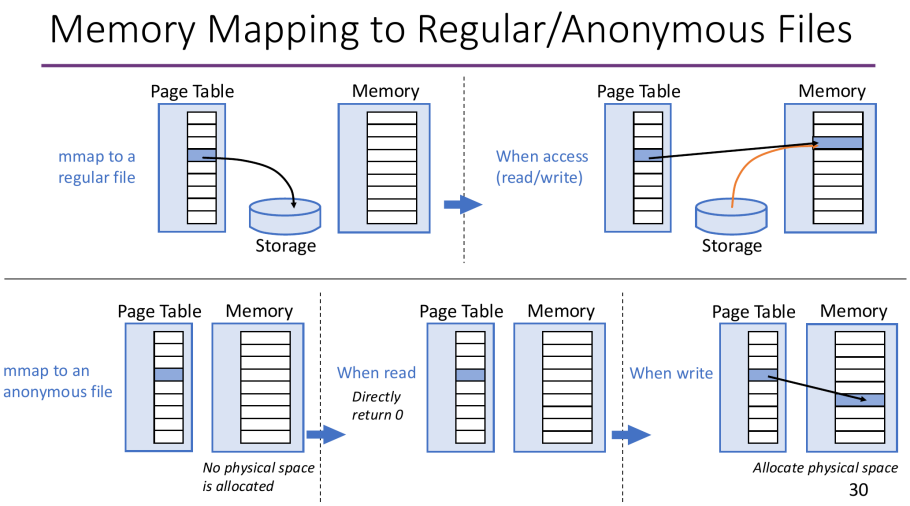



**Figure 16:** A regular file mapping initially records a link between virtual pages and storage; the first read or write brings data into memory. An anonymous mapping can return zeros on read without allocating a physical frame and allocate a frame only on the first write.

**execve.** Loading a new program discards the old user address space, creates a new page table, maps the executable file into code and initialized-data regions, creates anonymous mappings for stack and heap, and sets the program counter to the executable's entry point. Actual code and data pages are loaded lazily as page faults occur.

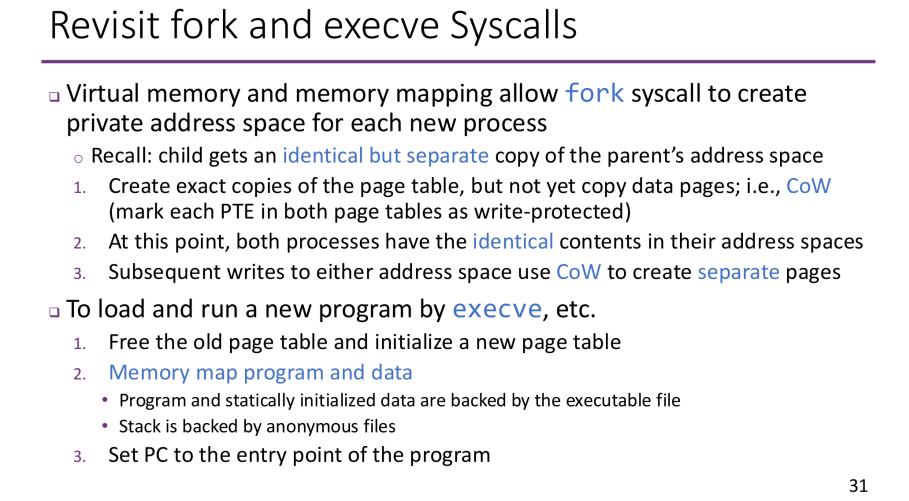



**Figure 17:** The fork system call uses copy-on-write page-table copying to create a private address space efficiently. The execve system call replaces the old page table, memory maps executable contents and anonymous regions, and starts execution at the program entry point.

## 9.5 User-level `mmap` and `munmap`

User programs can request memory mappings explicitly. A simplified POSIX-style interface is:

```
void *mmap(void *addr, size_t length, int prot,
           int flags, int fd, off_t offset);
int munmap(void *addr, size_t length);
```

The `addr` parameter is a preferred virtual address, often `NULL` to let the kernel choose. The `length` and `offset` describe the mapped range. The `prot` field requests permissions such as `PROT_READ` and `PROT_WRITE`. The `flags` field controls policies such as `MAP_PRIVATE`, `MAP_SHARED`, and `MAP_ANON`.

**Listing 1:** Mapping a file into virtual memory.

```c
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>

void read_file_with_mmap(const char *filename) {
    int fd = open(filename, O_RDONLY);
    if (fd < 0) {
        perror("open");
        exit(EXIT_FAILURE);
    }

    struct stat finfo;
    if (fstat(fd, &finfo) < 0) {
        perror("fstat");
        close(fd);
        exit(EXIT_FAILURE);
    }

    void *buf = mmap(NULL, (size_t)finfo.st_size,
                     PROT_READ, MAP_PRIVATE, fd, 0);
    if (buf == MAP_FAILED) {
        perror("mmap");
        close(fd);
        exit(EXIT_FAILURE);
    }

    /* The bytes of the file are now accessible through buf. */

    if (munmap(buf, (size_t)finfo.st_size) < 0) {
        perror("munmap");
    }
    close(fd);
}
```

# 10 Dynamic Memory Allocation

## 10.1 The heap and the allocator

The heap is the region of a process's virtual address space used for dynamic objects whose sizes or lifetimes are not known at compile time. In C, the standard interface is `malloc` and `free`; in C++, it is `new` and `delete`.

The allocator sits between the user program and the operating system. The user program asks for arbitrary byte sizes: 24 bytes, 100 bytes, an array of $n$ structures, and so on. The allocator satisfies these requests from heap memory it already owns. When it needs more heap space, it asks the kernel for more virtual pages, commonly through `mmap` or historically through `sbrk`.

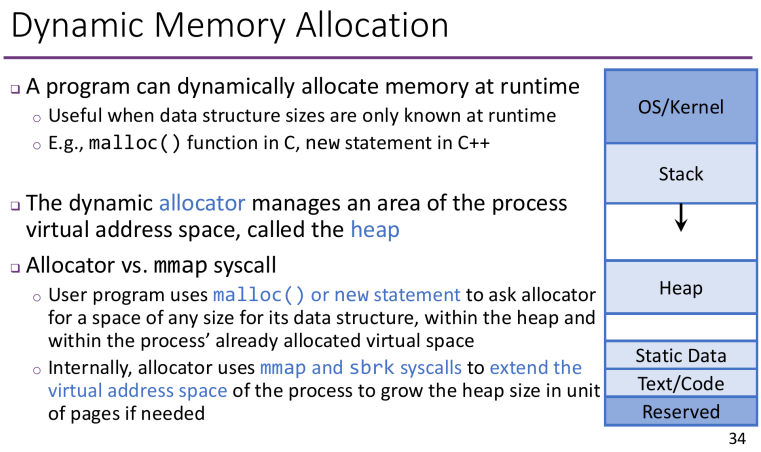



**Figure 18:** The dynamic allocator manages the heap, a region of the process virtual address space. User-level allocation requests are fine-grained, while the allocator obtains memory from the OS in page-sized units.

## 10.2 Allocator API

**Listing 2:** Basic explicit heap allocation in C.

```c
#include <stdio.h>
#include <stdlib.h>

int *make_array(size_t n) {
    int *p = malloc(n * sizeof(int));
    if (p == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }

    for (size_t i = 0; i < n; i++) {
        p[i] = (int)i;
    }
    return p;
}

int main(void) {
    size_t n = 100;
    int *a = make_array(n);

    /* Use a[0] ... a[n-1]. */

    free(a);
    return 0;
}
```

The contract is strict. The pointer passed to `free` must be a pointer previously returned by an allocation function and not already freed. Violating this contract causes undefined behavior.

## 10.3 Allocator internals

A typical allocator maintains metadata for each block: size, allocation status, and sometimes links to neighboring free blocks. When a request arrives, the allocator must find a free block large

enough, possibly split it, mark the allocated part as used, and return a pointer to the payload. When a block is freed, the allocator marks it free and may coalesce it with adjacent free blocks.

The allocator's design goals are in tension:

- **Speed**: allocation and free should usually take little time.
- **Space utilization**: the allocator should waste little memory.
- **Locality**: objects used together should ideally be placed near each other.
- **Concurrency**: multi-threaded programs need scalable allocation without excessive lock contention.

## 10.4 External fragmentation

External fragmentation occurs when free memory exists in aggregate but is split into pieces too small to satisfy a request. A first-fit allocator, which uses the first sufficiently large free block, can create this situation.

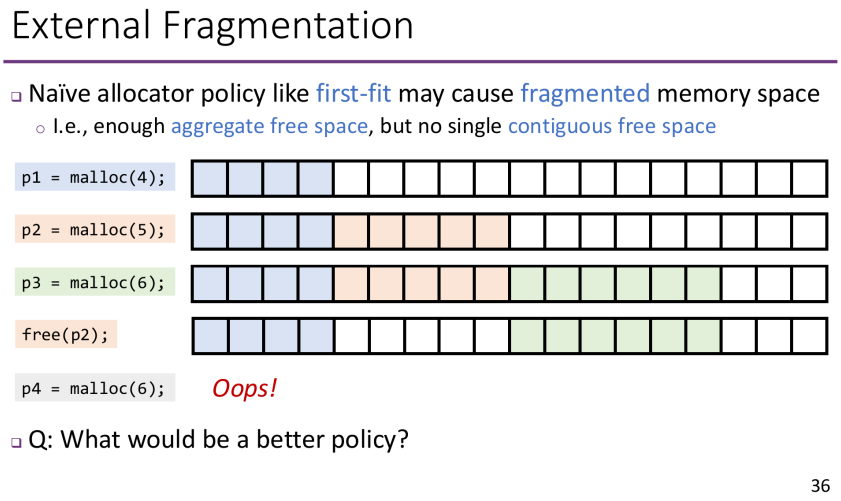



**Figure 19:** A simple first-fit allocator may suffer external fragmentation. After freeing a middle block, the total free capacity may be sufficient for a new request, but no single contiguous free block is large enough.

Coalescing neighboring free blocks helps, but it cannot solve fragmentation caused by allocated blocks that remain between free regions.

## 10.5 Segregated free lists

A common allocator design organizes free blocks into size classes, such as 1–2 bytes, 3–4 bytes, 5–8 bytes, 9–16 bytes, and so forth. Each class has its own free list. A request of size $n$ searches a size class that can hold $n$, then perhaps larger classes if needed.

**Algorithm 4:** Allocation with Segregated Free Lists

**Input:** Requested size $n$
**Output:** Pointer to a block of at least $n$ bytes

```
c ← smallest size class with block size at least n;
for each size class c, c + 1, ... do
    if the free list for this class is nonempty then
        remove one block b from the free list;
        if b is much larger than n then
            split b and return the remainder to an appropriate free list;
        mark b allocated;
        return pointer to b's payload;
request additional pages from the operating system;
insert new free memory into size-class lists;
retry allocation;
```

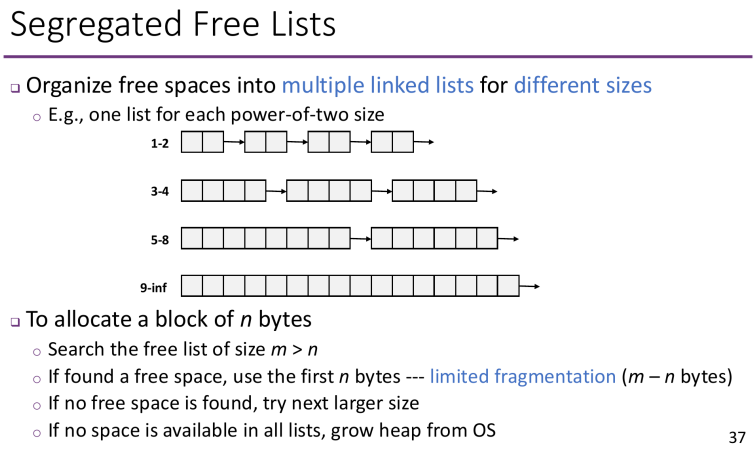



**Figure 20:** Segregated free lists group free blocks by size class. They reduce search cost and limit mismatch between requested and provided block sizes, but they require careful splitting, coalescing, and metadata management.

### 10.6 Internal fragmentation

Internal fragmentation is wasted space inside an allocated block. It arises when the allocator returns a block larger than requested, adds alignment padding, or stores metadata that is not part of the user's payload.

Good allocators balance internal fragmentation, external fragmentation, and speed. There is no universally optimal policy because workloads differ: a web server, a compiler, a database, and a scientific simulation allocate very different object patterns.

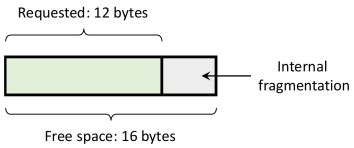
## 11 Implicit Memory Management and Garbage Collection

### 11.1 Why explicit memory management is error-prone

C and C++ give programmers explicit control over object lifetimes, but that control carries responsibility. Common memory-related bugs include bad pointer dereferences, allocating the wrong size, off-by-one writes, buffer overflows, incorrect pointer arithmetic, returning pointers to stack variables, use-after-free, leaks, and double frees.

## Internal Fragmentation

- Internal fragmentation happens when the requested size is smaller than the allocated free space, and the remaining space is wasted
- Caused by
  - Padding for alignment
  - Allocator policy, e.g., segregated lists
  - Metadata store, e.g., size, status
  - ……




- One solution: append remaining space of ($m - n$) to its corresponding free list, which can be used for a future small-size request
- Good allocators should be fast, and minimize both external and internal fragmentation

38

**Figure 21:** Internal fragmentation occurs when the allocated block is larger than the requested payload. The unused bytes are inside the allocated block and cannot be used by other requests unless the allocator splits the block.

## Why GC? – Common Memory-Related Bugs

- Dereference bad pointers

```
int val;
scanf("%d", val);
```

- Allocate wrong size

```
int **p;
p = malloc(N * sizeof(int));
for (int i = 0; i < N; i++) {
   p[i] = malloc(M * sizeof(int));
}
```

- Write unallocated memory (off-by-1)

```
int *p;
p = malloc(N * sizeof(int));
for (int i = 0; i <= N; i++) p[i] = i;
```

- Not checking the max length

```
char s[8];
gets(s);  // read 12345678 from input
```

- Misunderstanding pointer arithmetic

```
int *p;
while (p && *p != val) p += sizeof(int);
```

48

**Figure 22:** Examples of common C memory bugs: passing a value instead of an address to `scanf`, allocating the wrong amount of memory for a two-dimensional array, writing one element past the end, using unsafe input functions, and misunderstanding pointer arithmetic.

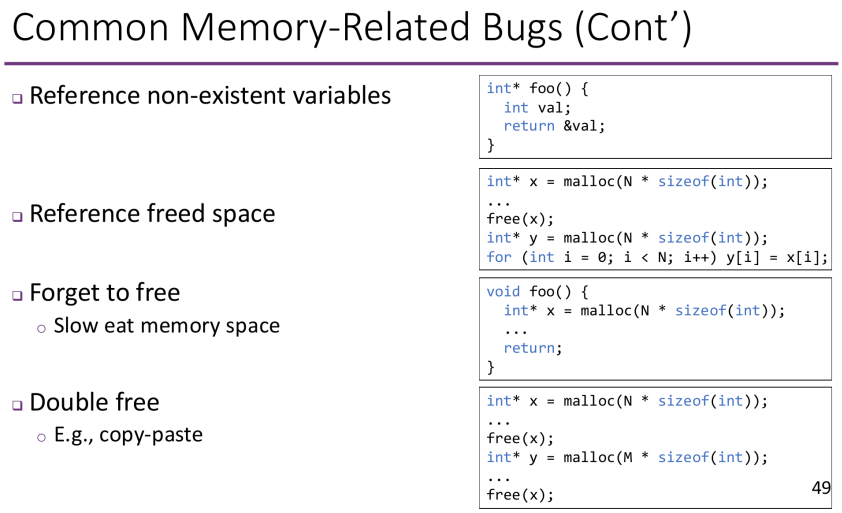



**Figure 23:** Additional common memory bugs: returning the address of a local variable, reading through a freed pointer, forgetting to free allocated memory, and freeing the same pointer twice.

Languages such as Java, Python, Perl, and Scala use implicit memory management: the runtime system automatically reclaims heap objects that can no longer be used. This process is called garbage collection.

**Definition**

**Garbage collection** is automatic reclamation of heap-allocated objects that are no longer reachable by the program.

## 11.2 Reachability and pointer discipline

The fundamental challenge is deciding when an object can be freed. In general, no runtime system can perfectly predict whether an object will be used in the future. Instead, garbage collectors use a conservative correctness rule: if no pointer can reach an object, then the program cannot use it, so it is safe to reclaim.

This requires the language or runtime to know which values are pointers. If a program can hide a pointer by casting it to an integer and later cast it back, the runtime might incorrectly believe the object is unreachable. Therefore, precise garbage collection relies on pointer discipline: pointers must be identifiable, and object layouts must reveal which fields contain references.

## 11.3 Memory as a graph

Garbage collection views memory as a directed graph. Heap objects are nodes. A pointer field from one object to another is an edge. Roots are references outside the heap, such as registers, stack slots, global variables, and runtime handles. Any heap object reachable from a root is live; unreachable heap objects are garbage.

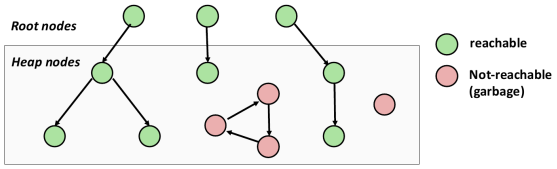
## 11.4 Mark-and-sweep garbage collection

Mark-and-sweep is a classical tracing collector. It has two phases:

1. **Mark**: start from all roots, traverse pointer edges, and set a mark bit in every reachable object.
2. **Sweep**: scan heap objects linearly and free every object whose mark bit is not set. Clear mark bits for the next cycle.

## Classical GC Algorithm: Mark-and-Sweep

- View memory as a graph, where each each object is a node
  - An edge exists if a pointer in an object points to another object
- Root nodes: objects that are not dynamically allocated in the heap
  - E.g., registers, locations on the stack, global variables, …
- Reachable nodes: heap objects that are reachable from any root node
- Non-reachable nodes are garbage to be collected




42

**Figure 24:** The object graph model of garbage collection. Root nodes outside the heap lead to reachable heap nodes. Heap nodes not reachable from any root cannot be accessed by the program and are garbage.

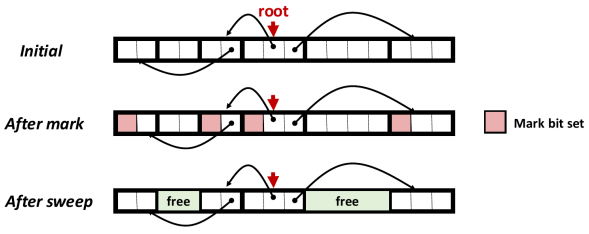
**Algorithm 5:** Mark-and-Sweep Garbage Collection

**Input:** Set of roots $R$, heap object set $H$
**Output:** Unreachable objects returned to the free pool

```
1 for each root r ∈ R do
2     if r points to an unmarked heap object then
3         recursively mark all objects reachable from r;
4 for each object o ∈ H do
5     if o.marked = 0 then
6         free o;
7     else
8         o.marked ← 0;
```

## Classical GC Algorithm 1: Mark-and-Sweep

- GC may start due to out-of-memory, or is periodically triggered
- Mark: start from root nodes, traverse graph and mark a bit in each reachable object; non-reachable objects are not visited and not marked
- Sweep: scan all objects in the memory, and free unmarked objects




43

**Figure 25:** Mark-and-sweep first marks reachable objects starting from roots, then sweeps through memory and frees unmarked objects. This can reclaim cyclic garbage but may pause program execution during collection.

The advantage of mark-and-sweep is that it correctly handles cycles: if a group of objects points only to each other but no root can reach them, none will be marked, so all can be collected. The disadvantage is that collection work may be delayed and may pause the program.

### 11.5 Reference counting

Reference counting stores, in each object, the number of references pointing to it. Assigning a pointer to an object increments its count; overwriting or destroying a pointer decrements the old object's count. When the count reaches zero, the object can be freed immediately.

**Algorithm 6:** Reference Counting Update on Pointer Assignment

**Input:** Pointer variable $p$, new target object $q$
**Output:** $p$ points to $q$ and unreachable zero-count objects are freed

```
1 old ← p;
2 if q ≠ NULL then
3     q.refcount ← q.refcount + 1;
4 p ← q;
5 if old ≠ NULL then
6     old.refcount ← old.refcount − 1;
7     if old.refcount = 0 then
8         recursively free old and decrement objects it references;
```

Reference counting reclaims many objects promptly and spreads overhead across normal execution. Its classic weakness is cycles. If $A$ points to $B$, $B$ points to $C$, and $C$ points back to $A$, each object may have a nonzero reference count even when no external root can reach the cycle. Additional cycle detection is needed to reclaim such structures.

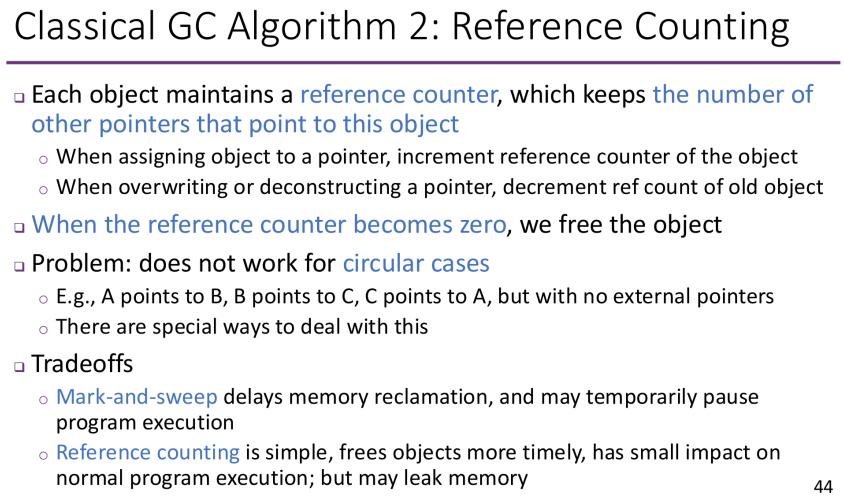



**Figure 26:** Reference counting maintains the number of pointers to each object and frees an object when the count becomes zero. It is simple and prompt but cannot by itself reclaim unreachable cycles.

## 12 Summary

Virtual memory is one of the most important examples of indirection in computer systems. By translating virtual addresses to physical addresses in page-sized units, it lets each process see a private linear address space while the operating system flexibly manages a shared physical memory resource.

The page table is the key structure that records mappings, permissions, and residency. Hardware uses it for translation and protection; the operating system manages it for allocation, paging, sharing, and fault handling. Demand paging treats DRAM as a cache for backing storage, and it relies on locality and working sets to perform well. The TLB is a hardware cache for

translations, avoiding page table accesses in the common case.

Large sparse virtual address spaces make flat page tables impractical, so modern systems use multi-level radix-tree page tables. This saves enormous space but increases the cost of TLB misses. Virtual memory also enables page sharing, copy-on-write, memory-mapped files, efficient `fork` and `execve`, and dynamic allocation through the heap.

At the user level, explicit allocators such as `malloc`/`free` must manage fragmentation and metadata efficiently. Higher-level languages often use garbage collection, which automatically reclaims unreachable heap objects using algorithms such as mark-and-sweep or reference counting. Across all levels, the recurring theme is the same: efficient memory systems combine abstraction, indirection, locality, and carefully engineered common-case performance.

# 13 Quizzes & Exercises

## 13.1 Short-Answer Quiz

1. What is the fundamental purpose of the virtual memory mechanism in modern computing?
2. Distinguish between a virtual address space and a physical address space.
3. What specific information is typically contained within a page table entry (PTE)?
4. Explain the role of indirection in virtual memory systems.
5. How does virtual memory serve as a tool for memory protection?
6. What occurs during a page fault, and how does the operating system resolve it?
7. Define a program's working set and explain what happens if it exceeds main memory capacity.
8. What is the translation look-aside buffer (TLB), and why is it necessary?
9. Why do modern systems such as x86-64 use multi-level page tables instead of a single-level table?
10. Describe the copy-on-write optimization and its benefit during process creation.

## 13.2 Essay Questions

1. **The locality principle in virtual memory.** Discuss how spatial and temporal locality justify the overhead of address translation and the use of DRAM as a software-managed cache for higher-capacity external storage.
2. **Anatomy of an address translation.** Trace the complete path of a memory request from the CPU's virtual address output, through the TLB and potential page table walks, to the final physical memory access, including permission-fault handling.
3. **The space-complexity challenge of page tables.** Analyze the tradeoffs involved in using multi-level page tables to manage sparse address spaces. How do they solve the size problem, and what new performance costs do they introduce?
4. **Memory management vs. memory protection.** Evaluate the dual role of the operating system and hardware in maintaining system stability. How does the page table structure simultaneously enable flexible memory allocation and rigorous process isolation?
5. **Dynamic memory allocation and fragmentation.** Compare explicit memory management with implicit garbage-collected memory management. Discuss how policies such as first-fit and segregated free lists balance speed against internal and external fragmentation.

## 13.3 Additional Practice Problems

1. A machine has 48-bit virtual addresses, 4-KiB pages, and 8-byte PTEs. Compute the number of virtual pages and the size of a flat page table. Explain why the result is

impractical.

2. Suppose a process sequentially scans a 1-MiB array using 4-KiB pages. How many distinct data pages does the scan touch? Why is a single TLB entry useful for many byte accesses within one page?
3. A process maps a file privately using `mmap` and then writes to one mapped page. Explain how this differs from a shared mapping and how copy-on-write may be involved.
4. In a mark-and-sweep collector, why are stack variables and registers considered roots? What could go wrong if the collector failed to scan one register containing the only pointer to a heap object?
5. Consider an allocator with size classes 16, 32, 64, and 128 bytes. For requests of 17 bytes, 31 bytes, and 65 bytes, which classes are used and how much internal fragmentation occurs before metadata overhead?

## 13.4 Answer Key for the Short-Answer Quiz

1. Virtual memory provides fine-grained, dynamic management of address spaces. It allows process address spaces to be physically non-contiguous, to grow or shrink at runtime, and to support controlled sharing and protection.
2. A virtual address space is the contiguous linear memory illusion provided to an individual process. The physical address space is the single hardware address space backed by actual DRAM and shared among all processes.
3. A PTE typically includes a valid/present bit, a physical frame number if resident, access permissions such as read/write/execute, and metadata such as dirty or accessed bits and backing-store information.
4. Indirection means that a process does not access physical memory directly. It names virtual addresses, and a translation layer maps those names to physical frames or backing storage, giving the OS flexibility and control.
5. Virtual memory isolates processes because each process has its own page table. If another process's data or kernel data is not mapped with appropriate permissions, the process cannot access it. Hardware checks permissions during translation.
6. A page fault is an exception caused by accessing a virtual page that is not currently resident or otherwise cannot be translated normally. The OS validates the access, brings the page from backing storage or creates a zero page if appropriate, updates the page table, and retries the instruction; invalid accesses are reported as errors.
7. A working set is the set of pages actively used over a time interval. If it fits in main memory, few page faults occur. If it exceeds memory capacity, the system may thrash by repeatedly swapping pages in and out.
8. The TLB is an on-chip cache of recent address translations. It is necessary because consulting the page table in memory for every access would be too slow.
9. Single-level page tables are too large for sparse 64-bit virtual address spaces. Multi-level page tables allocate translation nodes only for used regions, greatly reducing space overhead, at the cost of more expensive TLB misses.
10. Copy-on-write initially shares physical pages between a source and copy, marks them write-protected, and performs the actual copy only when a write occurs. This makes `fork` and other copying operations much cheaper when many pages are never modified.